

DevOps Lab Manual

Program 1: Create Project in Local and Remote Repository using Git

Aim:

To create a project in local repository and connect it to remote GitHub repository using Git Bash.

Procedure:

1. Install Git and configure username and email.
2. Create a local folder and initialize Git repository.
3. Create files and add content.
4. Add files to staging area and commit.
5. Create repository in GitHub.
6. Connect local repository to remote repository.
7. Push files to GitHub.

Commands:

- `git init`
- `git add .`
- `git commit -m 'Initial commit'`
- `git remote add origin URL`
- `git push -u origin main`

Output:

The commands executed successfully and expected output was observed.

Result:

Successfully created local and remote repository and pushed files to GitHub.

Create Project in Local and Remote Repository using Git and Git Bash

Step 1: Create a Local Project Folder

```
Mkdir MyProject  
cd MyProject
```

Step 2: Initialize Git Repository

```
git init
```

This creates a hidden .git folder that tracks your project.

Step 3: Create a Sample File

```
echo "Hello Git Project" > index.txt
```

Step 4: Add File to Staging Area

```
git add index.txt
```

To add all files:

```
git add .
```

Step 5: Commit the File

```
git commit -m "Initial commit"
```

Step 6: Create a Remote Repository

1. Go to GitHub
2. Click **New Repository**
3. Enter repository name (e.g., MyProject)
4. Click **Create Repository**
5. Copy the repository URL

Example:

```
https://github.com/username/MyProject.git
```

Step 7: Connect Local Repo to Remote Repo

```
git remote add origin https://github.com/username/MyProject.git
```

Check remote:

```
git remote -v
```

Step 8: Push Project to Remote Repository

```
git branch -M main
git push -u origin main
```

✓ Output

- Local repository created
- Remote repository created
- Files successfully pushed to remote repository

📋 Summary of Commands

Command	Description
git init	Initialize Git repository
git add .	Add files to staging
git commit -m "msg"	Commit files
git remote add origin URL	Link remote repo
git push -u origin main	Push files to remote

🎓 Viva Questions & Answers (Git – Local & Remote Repository)**1□.What is Git?****Answer:**

Git is a distributed version control system used to track changes in source code and manage collaborative software development.

2□.What is a repository?**Answer:**

A repository (repo) is a storage location where project files and their version history are maintained.

3□.What is the difference between local and remote repository?**Answer:**

- **Local repository:** Stored on your computer.

- **Remote repository:** Hosted on a server like GitHub and accessible over the internet.

4□.What does git init do?

Answer:

It initializes a new Git repository by creating a hidden .git directory in the project folder.

5□.What is the staging area?

Answer:

The staging area is an intermediate area where changes are prepared before committing them to the repository.

6□.What is the command to add files to staging?

Answer:

git add filename

or

git add .

7□.What is a commit?

Answer:

A commit is a snapshot of changes saved in the repository with a descriptive message.

8□.What does git commit -m "message" do?

Answer:

It saves staged changes permanently in the local repository with a commit message.

9□.What is a remote repository?

Answer:

A remote repository is a version of the project hosted online on platforms like GitHub, GitLab, or Bitbucket.

10. What does git remote add origin URL do?**Answer:**

It connects the local repository to a remote repository using a URL. Here, origin is the default name for the remote.

11. What is the purpose of git push?**Answer:**

It uploads local repository commits to the remote repository.

12. What does git branch -M main do?**Answer:**

It renames the current branch to main.

13. What is the difference between git pull and git push?**Answer:**

- git push → Uploads changes to remote repository.
- git pull → Downloads changes from remote repository.

14. What is a branch in Git?**Answer:**

A branch is a separate line of development in a repository.

15. Why is Git called a distributed version control system?**Answer:**

Because every developer has a complete copy of the repository, including its history.

16. What command is used to check remote repositories?**Answer:**

git remote -v

17. What is the default branch name in modern Git?**Answer:**

main

18. What happens if we do not commit before pushing?**Answer:**

Git will not push uncommitted changes. Only committed changes can be pushed.

19. What file stores Git configuration information?**Answer:**

The .git folder contains configuration and repository information.

20. What is the full form of Git?**Answer:**

Git is not officially an acronym. It was created by Linus Torvalds in 2005.

Program 2: Fork, Merge, Conflict and Rebase**Aim:**

To demonstrate fork, merge, conflict resolution and rebase using GitHub.

Procedure:

1. Fork repository from GitHub.
2. Clone forked repository.
3. Create new branch.
4. Make changes and commit.
5. Merge branch.
6. Resolve conflicts if any.
7. Perform rebase.

Commands:

- git clone URL
- git checkout -b branch
- git merge branch
- git rebase main

Output:

The commands executed successfully and expected output was observed.

Result:

Successfully performed fork, merge, conflict resolution and rebase.

1 □ Fork a Repository**★ What is Fork?**

Forking creates a copy of someone else's repository into your own account. It is commonly done on platforms like GitHub.

★ Steps to Fork (on GitHub)

1. Open any repository.
2. Click **Fork** (top right).
3. A copy will be created under your account.

Example:

Original repo → github.com/originaluser/project

Forked repo → github.com/yourusername/project

★ Clone the Forked Repository

```
git clone https://github.com/yourusername/project.git
cd project
```

2 □ Merge in Git**★ What is Merge?**

Merging combines changes from one branch into another branch.

★ Steps to Merge**Step 1: Create a new branch**

```
git checkout -b feature
```

Step 2: Make changes and commit

```
echo "Feature added" >> file.txt
git add .
git commit -m "Added feature"
```

Step 3: Switch to main branch`git checkout main`**Step 4: Merge feature branch**`git merge feature`**✓ Result:**

Changes from feature branch are now merged into main.

3□ Merge Conflict**✦ What is a Merge Conflict?**

A merge conflict occurs when:

- Two branches modify the same line of a file.
- Git cannot decide which change to keep.

✦ Example Conflict

If both branches modify the same line:

<<<<<<< HEAD

Hello from main branch

=====

Hello from feature branch

>>>>>>> feature

✦ Steps to Resolve Conflict

1. Open the file.
2. Remove conflict markers.
3. Edit the file manually.
4. Add and commit.

`git add file.txt``git commit -m "Resolved merge conflict"`**4□ Rebase in Git****✦ What is Rebase?**

Rebase moves or reapplies commits from one branch onto another branch. Instead of creating a merge commit, it creates a linear history.

✦ Merge vs Rebase

Merge	Rebase
Creates merge commit	No extra merge commit
Keeps branch history	Rewrites commit history
Safe for shared branches	Avoid on public/shared branches

✦ Steps to Rebase

Step 1: Switch to feature branch

git checkout feature

Step 2: Rebase onto main

git rebase main

✦ If conflict occurs during rebase:

git add file.txt

git rebase --continue

To abort rebase:

git rebase --abort

◆ Complete Workflow (Real Scenario)

1. Fork repository on GitHub
2. Clone fork
3. Create new branch
4. Make changes
5. Push branch
6. Create Pull Request
7. Merge changes

Summary of Important Commands

Command	Purpose
git clone URL	Clone repository
git checkout -b branch	Create new branch
git merge branch	Merge branch
git rebase branch	Rebase branch
git rebase --continue	Continue rebase
git rebase --abort	Cancel rebase

Program 2: Viva Questions & Answers

1□. What is Fork in Git?

Answer:

Fork is creating a copy of someone else's repository into your own account on platforms like GitHub so you can make changes independently.

2□. What is the difference between Fork and Clone?

Answer:

- **Fork:** Creates a copy of a repository on a remote server (like GitHub).
- **Clone:** Downloads a repository from a remote server to your local machine.

3□. What is Merge in Git?

Answer:

Merge is the process of combining changes from one branch into another branch.

4□. Which command is used to merge a branch?

Answer:

git merge branch_name

5□. What is a Merge Conflict?

Answer:

A merge conflict occurs when two branches modify the same line of a file and Git cannot automatically decide which change to keep.

6□. How do you resolve a merge conflict?**Answer:**

1. Open the conflicted file.
2. Remove conflict markers (<<<<<<<, =====, >>>>>>>).
3. Edit and save the correct content.
4. Run:

`git add filename``git commit -m "Resolved conflict"`**7□. What is Rebase in Git?****Answer:**

Rebase is the process of moving or reapplying commits from one branch onto another branch to maintain a linear project history.

8□. Which command is used to perform rebase?**Answer:**`git rebase branch_name`**9□. What is the difference between Merge and Rebase?****Answer:**

Merge	Rebase
Creates a merge commit	Does not create a merge commit
Preserves history	Rewrites history
Safe for shared branches	Avoid on public/shared branches

10. When should Rebase be avoided?**Answer:**

Rebase should be avoided on public/shared branches because it rewrites commit history, which can create problems for other collaborators.

Program 3: Interface Repository using Jenkins

Aim:

To automate project execution using Jenkins.

Procedure:

1. Install Jenkins.
2. Open Jenkins dashboard.
3. Create new job.
4. Connect GitHub repository.
5. Add build steps.
6. Run build.

Commands:

- Create Freestyle Project
- Add Git URL
- Build Now

Output:

The commands executed successfully and expected output was observed.

Result:

Successfully automated project execution using Jenkins.

1□. What is Jenkins?

Jenkins is an open-source automation server used for Continuous Integration (CI) and Continuous Deployment (CD).

It helps automate:

- Build
- Test
- Deployment

2□.Start Jenkins

After installation:

Open browser:

<http://localhost:8080>

Login with admin credentials.

3□. Install Required Plugins

Go to:

Manage Jenkins → Manage Plugins

Install:

- Git Plugin
- GitHub Plugin (if using GitHub)

4□. Create a New Jenkins Job

1. Click **New Item**
2. Enter Job Name (e.g., MyProject-Build)
3. Select **Freestyle Project**
4. Click **OK**

5□. Configure Git Repository in Jenkins

Under **Source Code Management:**

- Select **Git**
- Enter Repository URL:

<https://github.com/username/repository.git>

- Add credentials if repository is private.

6□. Configure Build Trigger (Optional)

Under **Build Triggers**, select:

✓ Build periodically

OR

✓ GitHub hook trigger for GITScm polling

This enables automatic build when code is pushed.

7□. Configure Build Step

Under **Build** → **Add Build Step**:

Select:

- Execute Shell (Linux/Mac)
- Execute Windows Batch Command (Windows)

Example:

echo "Build Started"

8□. Save and Build

Click:

- Save
- Build Now

Check:

- Console Output
- Build Status (Success/Failure)

◆ Workflow Summary

1. Push code to GitHub
2. Jenkins detects change
3. Jenkins pulls repository
4. Build is executed automatically

◆ Architecture Diagram (Conceptual Flow)

Developer → Git Repository → Jenkins → Build → Output

Important Jenkins Terms

Term	Meaning
Job	Task configured in Jenkins
Build	Execution of job
Workspace	Directory where project is built
Plugin	Extension to add functionality
CI	Continuous Integration

Program 3: Viva Questions & Answers

1□. What is Jenkins?

Answer:

Jenkins is an open-source automation server used for Continuous Integration (CI) and Continuous Deployment (CD). It automates build, test, and deployment processes.

2□.What is Continuous Integration (CI)?

Answer:

Continuous Integration is a software development practice where developers frequently integrate code into a shared repository, and each integration is automatically built and tested.

3□. What is the default port number of Jenkins?

Answer:

The default port number of Jenkins is **8080** (<http://localhost:8080>).

4□. How does Jenkins connect to a Git repository?

Answer:

Jenkins connects to a Git repository by configuring the repository URL under **Source Code Management** in a job and using Git plugins.

5□. Which plugin is required to connect GitHub with Jenkins?**Answer:**

The **Git Plugin** and optionally the **GitHub Plugin** are required to integrate repositories from GitHub.

6□. What is a Jenkins Job?**Answer:**

A Jenkins Job is a task or project configured in Jenkins to perform a specific operation like building or testing code.

7□. What is a Build in Jenkins?**Answer:**

A Build is the execution of a Jenkins job. It compiles code, runs tests, and performs defined steps.

8□. What is a Workspace in Jenkins?**Answer:**

Workspace is the directory in Jenkins where the project files are downloaded and the build process is executed.

9□. What is a Build Trigger?**Answer:**

A Build Trigger is a condition that automatically starts a build, such as:

- Code push to repository
- Scheduled time
- Webhook trigger

10. What is a Jenkins Pipeline?**Answer:**

A Jenkins Pipeline is a set of automated processes defined as code (using a Jenkinsfile) to manage the entire CI/CD workflow.

Program 4: Apply Scheduling in Jenkins

Aim:

To apply scheduling using Jenkins CI tool.

Procedure:

8. Open Jenkins job.
9. Configure build triggers.
10. Set schedule time.
11. Save and build.

Commands:

- H/2 * * * *
- H/5 * * * *

Output:

The commands executed successfully and expected output was observed.

Result:

Successfully scheduled builds using Jenkins.

1□. What is Scheduling in Jenkins?

Scheduling in Jenkins allows automatic execution of jobs at fixed time intervals using **Build Triggers**.

It is commonly used for:

- Nightly builds
- Weekly testing
- Regular backups
- Automated reports

2□. Steps to Apply Scheduling in Jenkins

Step 1: Open Jenkins Dashboard

Go to:

<http://localhost:8080>

Step 2: Open an Existing Job

Click on the job you created earlier.

Step 3: Click on Configure

Step 4: Enable Build Trigger

Under **Build Triggers**, select:

✓ **Build periodically**

Step 5: Enter Cron Expression

Jenkins uses Cron format:

MIN HOUR DOM MONTH DOW

Field	Meaning
MIN	Minutes (0–59)
HOUR	Hours (0–23)
DOM	Day of Month (1–31)
MONTH	Month (1–12)
DOW	Day of Week (0–7)

3□. Common Scheduling Examples

Run every 5 minutes

H/5 * * * *

Run every day at 9 AM

0 9 * * *

Run every Sunday at 10 PM

0 22 * * 0

Run once every hour

0 * * * *

4□. Save and Verify

1. Click **Save**
2. Jenkins will now automatically run the job as per schedule.
3. Check **Build History** to verify execution.

5□. Difference between Poll SCM and Build Periodically

Build Periodically	Poll SCM
Runs at fixed time	Checks repository for changes
Does not require code change	Triggers only if changes detected

6□. Real-Time Example Workflow

1. Developer pushes code to GitHub
2. Jenkins checks repository (Poll SCM)
3. Scheduled job runs automatically
4. Build and test executed

🔧 Advantages of Scheduling in Jenkins

- Fully automated builds
- Saves manual effort
- Ensures regular testing
- Improves CI/CD pipeline

Program 4: Viva Questions & Answers

1 □. What is Scheduling in Jenkins?

Answer:

Scheduling in Jenkins allows jobs to run automatically at specific time intervals using cron expressions under Build Triggers.

2 □. Which option is used to schedule a job in Jenkins?

Answer:

The Build periodically option under Build Triggers is used to schedule a job.

3 □. What format does Jenkins use for scheduling?

Answer:

Jenkins uses Cron expression format for scheduling jobs.

Format:

MIN HOUR DOM MONTH DOW

4 □. What is the default port number of Jenkins?

Answer:

The default port number of Jenkins is 8080.

5 □. Write a cron expression to run a job every day at 9 AM.

Answer:

0 9 * * *

6 □. What is the difference between Build Periodically and Poll SCM?

Answer:

Build Periodically	Poll SCM
Runs at fixed time	Checks repository for changes
Does not depend on code changes	Triggers only when changes are detected

7 □. What does “H” mean in Jenkins cron syntax?

Answer:

“H” (Hash) is used by Jenkins to distribute job execution times evenly to avoid overloading the system.

Example:

H/5 * * * *

(Runs every 5 minutes at hashed time)

8□. Where can you check scheduled build execution?

Answer:

Scheduled builds can be checked in the Build History section of the job dashboard.

9□. Can Jenkins schedule jobs without code changes?

Answer:

Yes, using Build periodically, Jenkins runs jobs even if there are no changes in the repository.

10. Why is scheduling important in CI/CD?

Answer:

Scheduling ensures:

- Regular automated builds
- Continuous testing
- Reduced manual effort
- Improved software quality

Program 5: Selenium WebdriverIO Automation Script

Aim:

To automate application using Selenium WebdriverIO.

Procedure:

1. Install NodeJS.
2. Install WebdriverIO.
3. Create test script.
4. Run script.

✂ Requirements

- Node.js installed
- npm (Node Package Manager)
- Web browser (Chrome/Firefox)
- WebdriverIO installed
- Basic knowledge of JavaScript

Commands:

- npm init
- npm install webdriverio
- npx wdio run wdio.conf.js

Output:

The commands executed successfully and expected output was observed.

Result:

Successfully automated application using Selenium WebdriverIO.

1□. What is Selenium?

Selenium is an open-source automation testing framework used for automating web applications.

It supports:

- Multiple browsers
- Multiple programming languages
- Cross-platform testing

2. What is WebdriverIO?

WebdriverIO is a JavaScript-based automation framework built on top of Selenium WebDriver protocol.

It is mainly used for:

- UI Automation Testing
- End-to-End Testing
- Integration Testing

3□. Install WebdriverIO

Open terminal:

```
npm init -y
```

```
npm install @wdio/cli --save-dev
```

```
npx wdio config
```

Follow configuration steps to generate wdio.conf.js.

4□. Sample Automation Script

```
describe('Google Search Test', () => {
  it('should open google and verify title', async () => {
    await browser.url('https://www.google.com');
    const title = await browser.getTitle();
    console.log("Page Title is: " + title);
  });
});
```

5□. Run the Test

```
npx wdio run wdio.conf.js
```

6□. Explanation of Script

Command	Purpose
browser.url()	Opens URL
browser.getTitle()	Gets page title
describe()	Test suite
it()	Test case

7□. Automation Workflow

1. Test script written in JavaScript
2. WebDriverIO sends commands to Selenium WebDriver
3. WebDriver interacts with browser
4. Browser executes actions
5. Results displayed in console

◆ Advantages

- Fast execution
- Supports modern JavaScript
- Easy configuration
- Cross-browser testing

🎓 Program 5: Viva Questions & Answers**1□. What is Selenium?**

Answer:

Selenium is an open-source tool used for automating web applications across different browsers and platforms.

2□. What is WebdriverIO?

Answer:

WebdriverIO is a JavaScript-based automation framework that uses the WebDriver protocol to automate browser interactions.

3□. Which programming language is mainly used in WebdriverIO?

Answer:

WebdriverIO primarily uses JavaScript (Node.js environment).

4□. What is the purpose of browser.url() command?

Answer:

browser.url() is used to open a specified website URL in the browser.

5□. What does browser.getTitle() do?

Answer:

It retrieves the title of the currently opened web page.

6□. What is the WebDriver protocol?

Answer:

WebDriver protocol is a standard communication protocol that allows automation scripts to control web browsers.

7□. What is the role of describe () and it () in WebdriverIO?

Answer:

- describe () defines a test suite.
- it () defines an individual test case.

8□. What is Cross-Browser Testing?

Answer:

Cross-browser testing ensures that a web application works correctly across different browsers like Chrome, Firefox, Edge, etc.

9□.What is the Page Object Model (POM)?

Answer:

Page Object Model is a design pattern in automation testing where web page elements and actions are stored in separate classes to improve maintainability.

10. What are the advantages of Selenium with WebdriverIO?

Answer:

- Supports multiple browsers
- Easy integration with CI/CD tools
- Supports modern JavaScript
- Scalable and maintainable automation framework

Program 6: XPath Locator Script

Aim:

To locate elements using different XPath types.

Procedure:

1. Open application.
2. Inspect elements.
3. Write XPath.
4. Execute script.

✂ Requirements

- Node.js installed
- WebdriverIO setup
- Browser (Chrome/Firefox)
- Basic HTML knowledge
- Selenium or WebdriverIO

Commands:

- `//input[@id='username']`
- `//button[text()='Login']`

Output:

The commands executed successfully and expected output was observed.

Result:

Successfully located elements using XPath.

1. What is XPath?

XPath (XML Path Language) is used to locate elements in an HTML or XML document.

It helps identify elements when:

- ID is not available
- Name is not unique
- Complex element structure exists

2□. Types of XPath

1. Absolute XPath

Starts from root node.

Example:

/html/body/div[1]/input

✗ Not recommended (breaks easily if structure changes)

2. Relative XPath

Starts from anywhere in the document.

Example:

//input[@id='username']

✓ Recommended

3□. Common XPath Syntax

Syntax	Description
//tagname	Selects all elements with tag
//tag[@attribute='value']	Select by attribute
//tag[text()='value']	Select by text
contains()	Partial match
starts-with()	Starts with value

4□. Example HTML

```
<input type="text" id="username" name="user">
```

```
<button id="loginBtn">Login</button>
```

5□. XPath Examples

Select username field:

```
//input[@id='username']
```

Select login button:

```
//button[text()='Login']
```

Using contains:

```
//button[contains(text(),'Log')]
```

6□. Sample WebdriverIO XPath Script

```
describe('XPath Locator Test', () => {
  it('should locate element using XPath', async () => {
    await browser.url('https://example.com');

    const username = await $('//input[@id="username"]');
    await username.setValue('admin');

    const loginBtn = await $('//button[text()="Login"]');
    await loginBtn.click();
  });
});
```

7□. Advantages of XPath

- Can navigate complex DOM structures
- Supports dynamic elements
- Flexible and powerful

8□. Disadvantages of XPath

- Slower than CSS selectors
- Absolute XPath is fragile
- Harder to read in complex cases

9□. Best Practices

- ✓ Prefer Relative XPath
- ✓ Use unique attributes
- ✓ Avoid long absolute paths
- ✓ Combine attributes for better accuracy

Example:

```
//input[@type='text' and @name='user']
```

🎓 Program 6: Viva Questions & Answers

1□. What is XPath?

Answer:

XPath (XML Path Language) is used to locate and navigate elements in an HTML or XML document during automation testing.

2□. Why is XPath used in Selenium?

Answer:

XPath is used when elements do not have unique IDs or names, and we need a flexible way to locate elements in the DOM using Selenium.

3□. What are the types of XPath?

Answer:

There are two types:

1. **Absolute XPath** – Starts from the root node.
2. **Relative XPath** – Starts from anywhere in the document (recommended).

4□. Which XPath is recommended and why?

Answer:

Relative XPath is recommended because it is shorter, more flexible, and less likely to break if the page structure changes.

5□. What is the syntax to select an element using attribute in XPath?**Answer:**

```
//tagname[@attribute='value']
```

Example:

```
//input[@id='username']
```

6□. What is contains() in XPath?**Answer:**

contains() is used for partial matching of attribute values or text.

Example:

```
//button[contains(text(),'Log')]
```

7□. How do you select an element using text in XPath?**Answer:**

```
//tagname[text()='value']
```

Example:

```
//button[text()='Login']
```

8□. What is the difference between XPath and CSS Selector?**Answer:**

XPath	CSS Selector
Can navigate forward and backward in DOM	Only forward traversal
More powerful and flexible	Faster and simpler
Supports text() function	Does not support text matching directly

9□. What are the advantages of XPath?**Answer:**

- Can handle dynamic elements
- Works when ID or name is not available
- Supports complex DOM navigation

11. What are the disadvantages of XPath?**Answer:**

- Slower than CSS selectors
- Absolute XPath is fragile
- Can become complex and hard to maintain

Program 7: Maven Lifecycle Commands

Aim:

To apply Maven lifecycle commands and integrate with Jenkins.

Procedure:

1. Install Maven.
2. Create project.
3. Run lifecycle commands.
4. Integrate with Jenkins.

✂ Requirements

- Java installed
- Maven installed
- Basic knowledge of Java project structure

Commands:

- mvn compile
- mvn test
- mvn package
- mvn install

Output:

The commands executed successfully and expected output was observed.

Result:

Successfully executed Maven lifecycle and integrated with Jenkins.

1□. What is Maven?

Apache Maven is a build automation and project management tool used primarily for Java projects.

It manages:

- Project build
- Dependencies
- Documentation
- Testing
- Packaging

2□. What is Maven Lifecycle?

A Maven lifecycle is a sequence of phases used to build and deploy a project.

There are **three main lifecycles**:

Lifecycle	Purpose
Default	Build and deploy project
Clean	Remove build files
Site	Generate project documentation

3□. Default Lifecycle Phases

Important phases:

1. **validate** – Checks project correctness
2. **compile** – Compiles source code
3. **test** – Runs test cases
4. **package** – Creates JAR/WAR file
5. **verify** – Runs checks
6. **install** – Installs package to local repository
7. **deploy** – Deploys to remote repository

4□. Clean Lifecycle

Phase	Purpose
pre-clean	Tasks before cleaning
clean	Deletes target folder
post-clean	Tasks after cleaning

Command:

mvn clean

5□. Site Lifecycle

Used to generate project documentation.

Command:

mvn site

6□. Common Maven Commands**Compile Project**

mvn compile

Run Tests

mvn test

Package Project

mvn package

Install to Local Repository

mvn install

Clean and Package

mvn clean package

7□. What is POM.xml?

POM stands for **Project Object Model**.

It is an XML file that contains:

- Project details
- Dependencies
- Plugins
- Build configurations

8□. Maven Directory Structure

```
src/  
├── main/  
│   └── java/  
├── test/  
│   └── java/
```

target/
pom.xml

9□. Example Workflow

1. Write Java code
2. Run:

mvn clean install

3. Maven:
 - Cleans old build
 - Compiles code
 - Runs tests
 - Creates JAR
 - Installs in local repository

11. Advantages of Maven

- Automatic dependency management
- Standard project structure
- Easy build process
- Integration with Jenkins

🎓 Program 7: Viva Questions & Answers

1□. What is Maven?

Answer:

Apache Maven is a build automation and project management tool mainly used for Java projects to manage dependencies, build, and deployment.

2□. What is a Maven Lifecycle?

Answer:

A Maven lifecycle is a sequence of predefined phases that define the build process of a project.

3□. What are the three main Maven lifecycles?**Answer:**

The three main lifecycles are:

1. **Default** – Builds and deploys the project
2. **Clean** – Removes previous build files
3. **Site** – Generates project documentation

4□. What is the purpose of mvn clean?**Answer:**

mvn clean deletes the target directory and removes previously compiled files.

5□. What does mvn compile do?**Answer:**

mvn compile compiles the source code of the project.

6□. What is the use of mvn test?**Answer:**

mvn test runs the test cases present in the project.

7□. What does mvn package do?**Answer:**

mvn package compiles the code, runs tests, and packages the application into a JAR or WAR file.

8□. What is POM.xml?**Answer:**

POM (Project Object Model) is an XML file that contains project configuration details such as dependencies, plugins, and build settings.

9□. What is the purpose of mvn install?**Answer:**

mvn install installs the built package into the local Maven repository so other projects can use it.

11. What is the difference between install and deploy?**Answer:****Install**

Installs artifact in local repository

Used for local development

Deploy

Uploads artifact to remote repository

Used for sharing with team

Program 8: Docker Commands

Aim:

To apply Docker commands and integrate with tools.

Procedure:

1. Install Docker.
2. Pull image.
3. Run container.
4. Stop container.

✂ Requirements

- Docker installed
- Basic knowledge of command line
- Internet connection (to pull images)

Commands:

- `docker pull nginx`
- `docker run nginx`
- `docker ps`

Output:

The commands executed successfully and expected output was observed.

Result:

Successfully executed Docker commands.

1□. What is Docker?

Docker is an open-source containerization platform used to build, ship, and run applications in containers.

It allows applications to run consistently across different environments.

2□. What is a Container?

A container is a lightweight, standalone, executable package that includes:

- Application code
- Runtime
- Libraries
- Dependencies

3□. What is an Image?

A Docker Image is a read-only template used to create containers.

Example:

```
docker pull ubuntu
```

4□. Basic Docker Commands

1. Check Docker Version

```
docker --version
```

2. Pull an Image

```
docker pull nginx
```

3. List Images

```
docker images
```

4. Run a Container

```
docker run nginx
```

Run in detached mode:

```
docker run -d nginx
```

5. List Running Containers

```
docker ps
```


List all containers:

```
docker ps -a
```

6. Stop a Container

```
docker stop container_id
```

7. Remove a Container

```
docker rm container_id
```

8. Remove an Image

```
docker rmi image_id
```

9. Execute Command Inside Container

```
docker exec -it container_id /bin/bash
```

10. Build Image from Dockerfile

```
docker build -t myimage .
```

5□. Example Workflow

1. Pull image from Docker Hub
2. Run container
3. Check running containers
4. Stop container
5. Remove container

6□. Important Docker Terms

Term	Meaning
Image	Blueprint for container
Container	Running instance of image
Dockerfile	Script to build image
Docker Hub	Public image repository
Volume	Persistent storage

7□. Advantages of Docker

- Lightweight
- Fast deployment
- Environment consistency
- Easy scaling
- Works well with CI/CD tools

🎓 Program 8: Viva Questions & Answers

1. What is Docker?

Answer:

Docker is an open-source containerization platform used to develop, ship, and run applications inside lightweight containers.

2□. What is a Docker Image?

Answer:

A Docker Image is a read-only template that contains application code, libraries, and dependencies required to run an application.

3□. What is a Docker Container?

Answer:

A Docker Container is a running instance of a Docker image.

4□. Which command is used to check Docker version?

Answer:

```
docker --version
```

5□. How do you pull an image from Docker Hub?

Answer:

```
docker pull image_name
```

Example:

```
docker pull nginx
```

6□. How do you list running containers?**Answer:**`docker ps`

To list all containers:

`docker ps -a`**7□. How do you stop a running container?****Answer:**`docker stop container_id`**8□. What is the difference between Docker Image and Container?****Answer:**

Docker Image	Docker Container
Blueprint/template	Running instance
Read-only	Executable
Used to create containers	Created from image

9□. What is Dockerfile?**Answer:**

A Dockerfile is a text file that contains instructions to build a Docker image automatically.

11. What are the advantages of Docker?**Answer:**

- Lightweight and fast
- Platform independent and Easy deployment
- Consistent environment
- Supports CI/CD integration

Program 9: Backlogs, Sprints and jQuery

Aim:

To create tasks and retrieve using jQuery.

Procedure:

12.Create HTML file.

13.Add tasks.

14.Use jQuery.

15.Retrieve tasks.

Commands:

- `$('#tasks li')`

Output:

The commands executed successfully and expected output was observed.

Result:

Successfully retrieved tasks using jQuery.

PART 1: Backlogs and Sprints (Agile Methodology)

1□. What is Agile?

Agile is a software development methodology that promotes iterative development, collaboration, and continuous improvement.

One of the most popular Agile frameworks is Scrum.

2□. What is a Product Backlog?

A **Product Backlog** is a prioritized list of features, enhancements, bug fixes, and tasks required for a project.

It is managed by the **Product Owner**.

Example:

- User login feature
- Payment integration
- Bug fixes
- UI improvement

3□. What is a Sprint?

A **Sprint** is a fixed time period (usually 2–4 weeks) during which selected backlog items are completed.

At the end of a Sprint:

- A working product increment is delivered.

4□. Sprint Lifecycle

1. Sprint Planning
2. Development
3. Daily Standup
4. Sprint Review
5. Sprint Retrospective

5□. Tools Used for Backlogs & Sprints

Common Agile tools:

- Jira
- Trello

- Azure DevOps

PART 2: jQuery

6□. What is jQuery?

jQuery is a fast and lightweight JavaScript library used to simplify HTML DOM manipulation, event handling, and animations.

7□. Why Use jQuery?

- Simplifies JavaScript code
- Easy DOM manipulation
- Cross-browser compatibility
- AJAX support

8□. Adding jQuery to HTML

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

9□. Basic jQuery Example

HTML:

```
<button id="btn">Click Me</button>
<p id="text">Hello World</p>
```

jQuery Script:

```
$(document).ready(function(){
    $("#btn").click(function(){
        $("#text").hide();
    });
});
```

11. Explanation

Command	Meaning
\$(document).ready()	Runs code after page loads
\$("#id")	Select element by ID
.click()	Click event
.hide()	Hides element

◆ Real-Time Agile + jQuery Example

1. Backlog item: "Add login button animation"
2. Sprint: 2 weeks
3. Developer implements using jQuery animation

Example:

```
$("#btn").click(function(){
    $("#text").fadeOut();
});
```

📖 Program 9: Viva Questions & Answers

◆ Backlogs & Sprints (Agile)

1□. What is a Product Backlog?

Answer:

A Product Backlog is a prioritized list of features, enhancements, bug fixes, and tasks required to complete a project in Agile methodology.

2□. What is a Sprint?

Answer:

A Sprint is a fixed time period (usually 2–4 weeks) in which selected backlog items are developed and delivered as a working product increment.

3□. Who manages the Product Backlog?

Answer:

The Product Backlog is managed by the **Product Owner** in Scrum.

4□.What happens during Sprint Planning?**Answer:**

During Sprint Planning, the team selects backlog items and defines the tasks to be completed in the upcoming sprint.

5□. Name any tools used for managing Backlogs and Sprints.**Answer:**

Some popular tools are:

- Jira
- Trello
- Azure DevOps

◆ jQuery**6□. What is jQuery?****Answer:**

jQuery is a lightweight JavaScript library used to simplify HTML DOM manipulation, event handling, animations, and AJAX operations.

7□. What is the purpose of \$(document).ready()?**Answer:**

\$(document).ready() ensures that the JavaScript code runs only after the entire webpage has fully loaded.

8□. How do you select an element by ID in jQuery?**Answer:**

Using # symbol.

Example:

```
$("#elementId")
```

9□. What is the use of .click() function in jQuery?**Answer:**

The .click() function is used to attach a click event handler to an element.

11. What are the advantages of jQuery?**Answer:**

- Simplifies JavaScript coding
- Easy DOM manipulation
- Cross-browser compatibility
- Supports animations and AJAX

Program 10: DevOps Framework Integration

Aim:

To integrate DevOps tools in framework.

Procedure:

1. Create Git repository.
2. Connect Jenkins.
3. Build using Maven.
4. Test using Selenium.
5. Deploy using Docker.

Commands:

- git push
- mvn install
- docker run

Output:

The commands executed successfully and expected output was observed.

Result:

Successfully integrated DevOps tools.

1□. What is DevOps?

DevOps is a combination of Development (Dev) and Operations (Ops) that promotes collaboration, automation, and continuous delivery of software.

It focuses on:

- Continuous Integration (CI)
- Continuous Deployment (CD)
- Automation
- Monitoring

2□. DevOps Lifecycle

Typical DevOps lifecycle stages:

1. Plan
2. Develop
3. Build
4. Test
5. Release
6. Deploy
7. Operate
8. Monitor

3. Tools Used in DevOps Integration

Stage	Tool Example
Version Control	Git
Build Tool	Apache Maven
CI/CD	Jenkins
Containerization	Docker
Orchestration	Kubernetes
Monitoring	Prometheus

4□. DevOps Integration Workflow

Step 1: Developer pushes code

Code is pushed to repository using Git.

Step 2: Continuous Integration

Jenkins automatically pulls the code and builds using Maven.

Step 3: Testing

Automated tests are executed.

Step 4: Containerization

Application is packaged using Docker.

Step 5: Deployment

Docker container is deployed using Kubernetes.

Step 6: Monitoring

Prometheus monitors performance and logs.

5□. Example CI/CD Flow

Developer → Git → Jenkins → Maven → Docker → Kubernetes → Monitoring

6□. Benefits of DevOps Integration

- Faster software delivery
- Reduced manual errors
- Continuous testing
- Improved collaboration
- High system reliability

7□. Key DevOps Concepts

Concept	Meaning
CI	Continuous Integration
CD	Continuous Deployment/Delivery
Pipeline	Automated workflow
Infrastructure as Code	Managing infrastructure using code
Automation	Reducing manual work

8□. Real-Time Example

1. Code committed to Git
2. Jenkins pipeline triggered
3. Maven builds project
4. Docker image created
5. Kubernetes deploys container
6. Monitoring tools track performance

Example Jenkins Pipeline Script:

```

pipeline {
    agent any
    stages {
        stage('Clone') {
            steps {
                git 'https://github.com/username/repo.git'
            }
        }
        stage('Build') {
            steps {
                bat 'mvn clean install'
            }
        }
        stage('Test') {
            steps {
                bat 'npm test'
            }
        }
        stage('Deploy') {
            steps {
                bat 'docker run -d myapp'
            }
        }
    }
}

```

🎓 Program 10: Viva Questions & Answers**1□. What is DevOps?****Answer:**

DevOps is a methodology that combines Development and Operations to automate and improve the software development lifecycle through Continuous Integration and Continuous Deployment.

2□. What is Continuous Integration (CI)?**Answer:**

Continuous Integration is the practice of automatically building and testing code whenever changes are committed to a version control system like Git.

3□. What is Continuous Deployment (CD)?**Answer:**

Continuous Deployment is the process of automatically deploying tested code to production without manual intervention.

4□. Which tool is commonly used for CI/CD automation?**Answer:**

Jenkins is widely used for automating CI/CD pipelines.

5□. What is the role of Docker in DevOps?**Answer:**

Docker is used to package applications and their dependencies into containers for consistent deployment across environments.

6□. What is Kubernetes used for?**Answer:**

Kubernetes is used to manage, scale, and deploy containerized applications.

7□. What is Infrastructure as Code (IaC)?**Answer:**

Infrastructure as Code is the practice of managing and provisioning infrastructure using configuration files instead of manual setup.

8□. What is a DevOps Pipeline?**Answer:**

A DevOps pipeline is an automated workflow that moves code from development to production through build, test, and deployment stages.

9□. What are the benefits of DevOps integration?**Answer:**

- Faster delivery
- Improved collaboration
- Reduced errors
- Continuous monitoring
- Better software quality

11. How does DevOps improve software quality?**Answer:**

DevOps improves quality by enabling automated testing, continuous monitoring, quick feedback, and faster issue resolution.